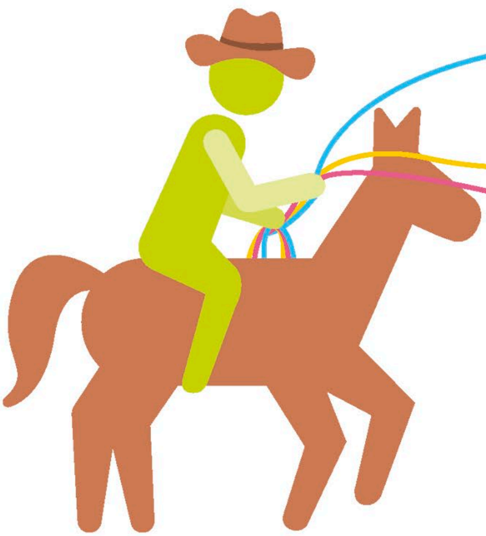
Loops

Programs often need to repeat the same task multiple times. To avoid re-writing code, programmers use loops. Once a program reaches the end of a looping structure, it goes back to the beginning of the loop and starts again. There are three types of loops – FOR, WHILE, and DO-WHILE. Choosing the right one depends on the duration of the loop, the elements a user wants to change in each iteration, and how the user wants to exit the loop.

FOR loops

This loop runs a block of code a fixed number of times. If you wanted to draw a square, instead of coding for each side to be drawn, you could write the code to draw one line and then turn 90 degrees, and run this four times in a FOR loop.



WHILE loops

This is equivalent to saying "loop forever, on one condition". This condition could be "WHILE there are still biscuits in the biscuit tin: don't buy new biscuits" or "until the user closes the program: keep recording key presses".

DO-WHILE loops

Similar to WHILE loops, DO-WHILE loops execute for an indefinite amount of time. The only difference is that this loop checks its condition at the end, so it's guaranteed to be executed at least once. For example, if a user inputs "11" into a program asking for a number between 1 and 10, the answer will be rejected and the user will be asked for a value again.

Why use functions?

Functions separate what the code does from how it acts. A function takes input data, such as numbers or coordinates, and turns it into output data, such as an answer or a full address. On the inside, functions use variables, constants, arrays, and control structures such as loops and branches. They help make code more readable.

▽ **Fahrenheit to Celsius**
A function that takes a temperature in Fahrenheit and outputs the temperature in Celsius eliminates the potential for human error as a human does not need to do the calculation.